SPURS² HACKATHON · OCT 22–23, 2026

Build with TwelveLabs

Your quickstart to search, analysis, and agents for video

MARENGO PEGASUS JOCKEY

docs.twelvelabs.io · playground.twelvelabs.io

THE PLATFORM AT A GLANCE

Upload once. Search, analyze, or ask.

MARENGO · EMBED + SEARCH

Find it

Turns video, audio, images, and text into embeddings, so you can search footage in plain language.

PEGASUS · ANALYZE

Describe it

Watches a video and writes text: summaries, chapters, answers, JSON, or timed segments.

JOCKEY · AGENT

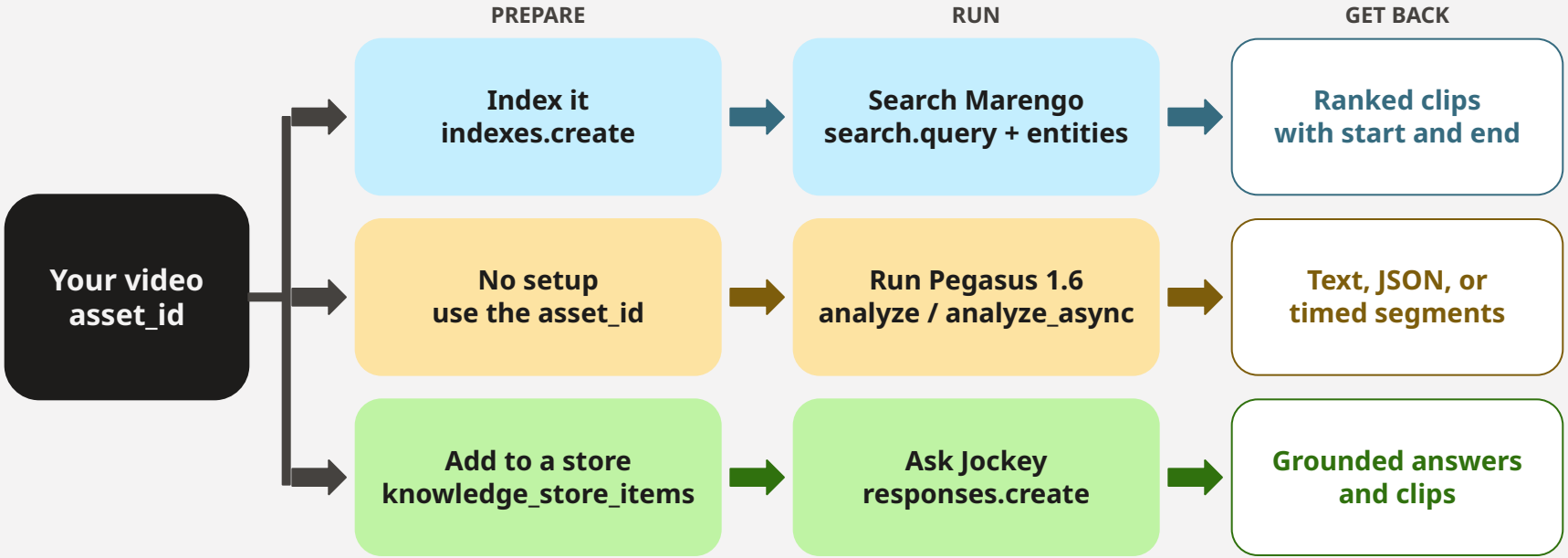
Ask it

An agent that plans its own steps across your whole library and answers with cited moments.

Models API: you choose exactly what runs. Agents API (Jockey): the agent picks the steps for you.

HOW IT FITS TOGETHER

One asset, three ways to use it



STEP 0 · PLAYGROUND

Test it in the Playground before you code

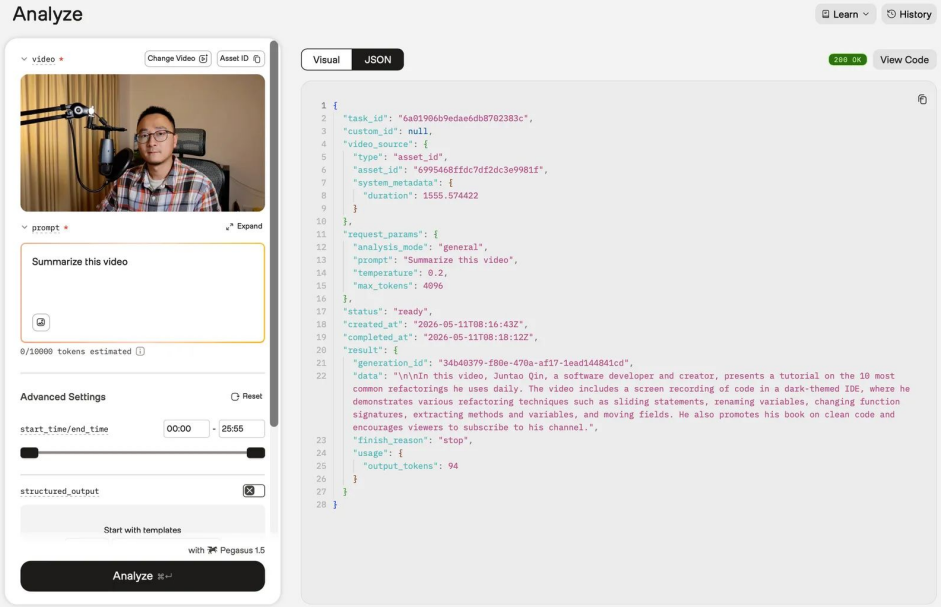
playground.twelvelabs.io

1 Upload videos and images in Assets

2 Run Search, Analyze, or Segment

3 Flip Visual to JSON for raw output

4 View Code copies the exact request



STEP 0 · API KEY

Get your API key and SDK in 2 minutes

1 Sign up

Free account at playground.twelvelabs.io

2 Create a key

Settings → API Keys → Create API Key. Copy it.

3 Install the SDK

Python or Node.js. Always install the latest version.

```
pip install --upgrade twelvelabs        # Python
npm install twelvelabs-js@latest        # Node.js
client = TwelveLabs(api_key=os.environ["TWELVELABS_API_KEY"])
```

Calling REST directly? Send the key in the x-api-key header to https://api.twelvelabs.io/v1.3

STEP 1 · ASSETS

Assets: upload once, reuse everywhere

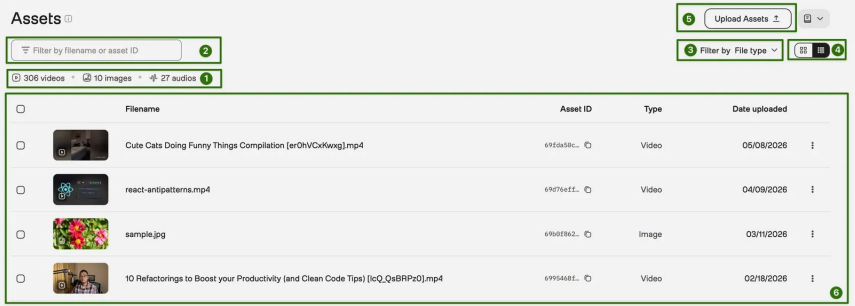
Public URL · up to 4 GB

Local file · up to 200 MB

Multipart · up to 10 GB

Then pass the asset_id to:

- Indexes → Marengo search
- Analyze → Pegasus text
- Embed → Marengo vectors
- Entities → player photos
- Knowledge stores → Jockey



Processing is async: poll until status is "ready". Use direct media links. YouTube, Drive, and Dropbox links fail.

PART 1 · FIND IT

Marengo

Embeddings, indexes, search, and entity search

MARENGO · EMBEDDING MODEL

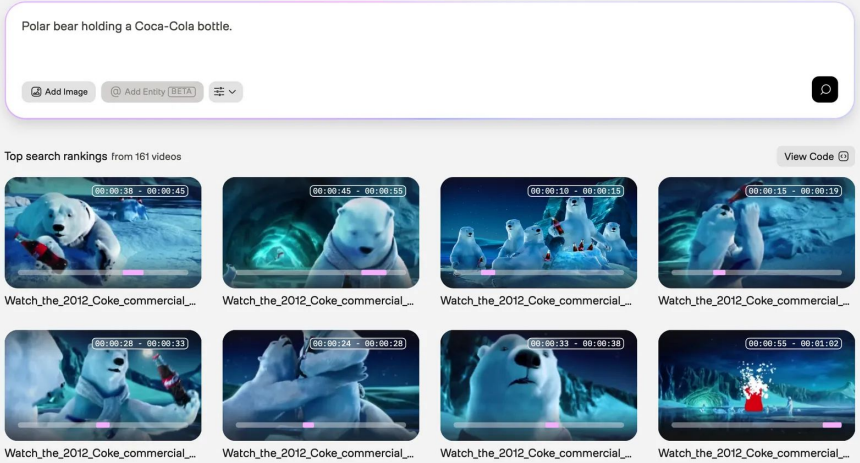
Marengo turns video into searchable meaning

It watches the visuals, listens to speech and sound, and reads on-screen text. Each moment becomes an embedding (a vector), so similar moments sit close together.

Marengo 3.0: powers search inside an index

Marengo 3.5: embeddings you store yourself (128/256/512 dims)

Use it for semantic search, similar-play lookup, clustering, and RAG.



One plain-language query matches a character, an object, a logo, and an action.

MARENGO · INDEX + SEARCH

Build an index, then search it

1 Create an index

2 Add your asset

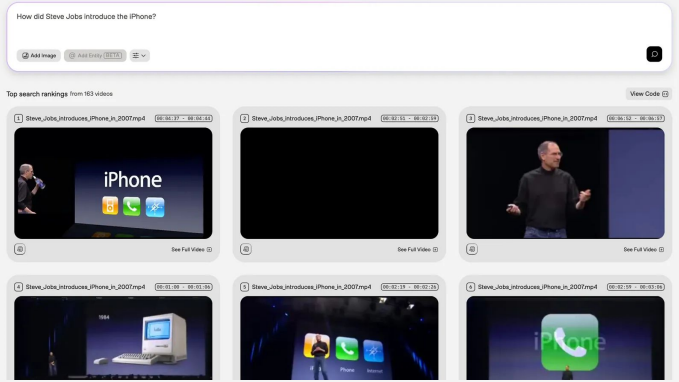
3 Wait for "ready"

4 Search

```
index = client.indexes.create(
  index_name="spurs-games",
  models=[{"model_name": "marengo3.0",
           "model_options": ["visual", "audio"]}])

client.indexes.indexed_assets.create(
  index_id=index.id, asset_id=asset.id)

hits = client.search.query(index_id=index.id,
  query_text="corner three in transition",
  search_options=["visual", "audio"])
```



Each hit returns video_id, start, end (seconds), and rank (1 = best).

MARENGO · ENTITY SEARCH (BETA)

Find a specific player with entities

1 Create a collection

One per roster, e.g. "Spurs"

2 Create an entity

A player + reference photos

3 Search with <@entity_id>

Mention the player in the query

```
roster = client.entity_collections.create(name="Spurs")

player = client.entity_collections.entities.create(
  entity_collection_id=roster.id,
  name="Player 1",
  asset_ids=[photo1.id, photo2.id, photo3.id])

hits = client.search.query(index_id=index.id,
  query_text=f"<@{player.id}> dunks on a fast break",
  search_options=["visual", "audio"])
```

Tips

- Upload several clear photos per player: different angles, lighting, and jerseys
- Keep each team in its own collection for better accuracy
- Works on any index with Marengo enabled

PART 2 · DESCRIBE IT

Pegasus 1.6

General mode, structured JSON, and timed segments

PEGASUS · VIDEO-LANGUAGE MODEL

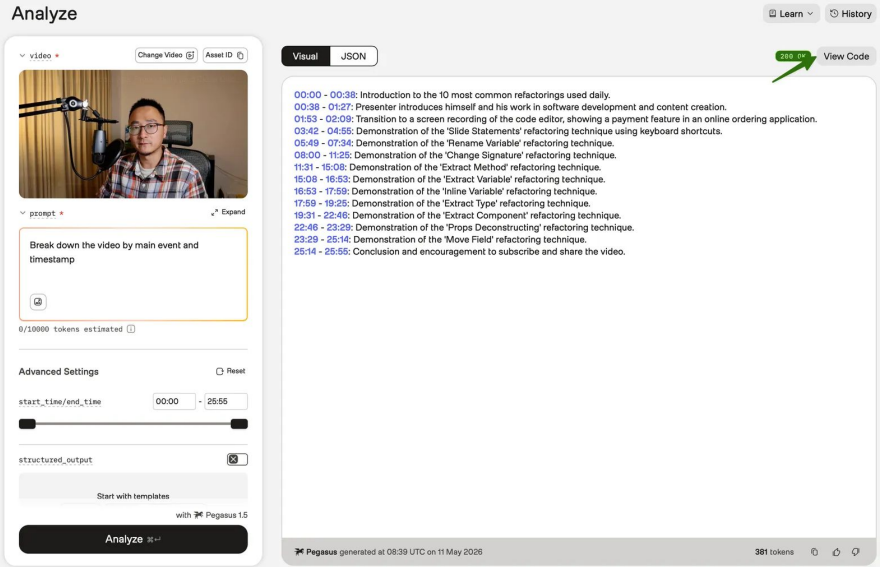
Pegasus watches video and writes about it

Give it a video (or up to 20 images) and a prompt. It answers in text grounded in what it sees and hears.

General mode: prompt in, text or JSON out

Time-based metadata: timed segments out

Use pegasus1.6. Sync calls handle videos up to 1 hour; async tasks go longer.



PEGASUS · GENERAL MODE

General mode: the parameters that matter

`model_name`	Use "pegasus1.6". Omitting it falls back to pegasus1.5.
`video`	What to watch: an asset_id, a public URL, or base64.
`prompt`	Your instruction or question. Use prompt_v2 to reference images.
`temperature`	Randomness from 0 to 1. Default 0.2. Use 0 for repeatable output.
`max_tokens`	Cap on response length. Default 4,096 tokens.
`response_format`	Add a JSON schema to get structured output (next slide).
`start_time / end_time`	Analyze only part of the video, in seconds.
`stream`	Sync only: stream text as it is generated. Default true.

PEGASUS · STRUCTURED OUTPUT

Get JSON back with response_format

```
res = client.analyze(
  model_name="pegasus1.6",
  video=VideoContext_AssetId(asset_id=asset.id),
  prompt="List every basket and who scored.",
  response_format=SyncResponseFormat(
    type="json_schema",
    json_schema={"type": "object", "properties": {
      "plays": {"type": "array", "items": {
        "type": "object", "properties": {
          "player": {"type": "string"},
          "start_time": {"type": "number"}}}}}))

data = json.loads(res.data)
```

The prompt says what to extract. The JSON schema sets the shape for the output.

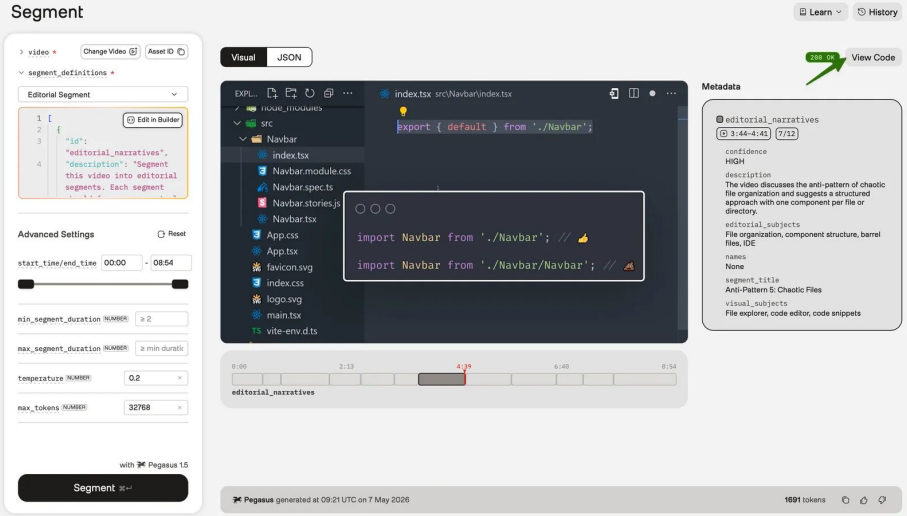
res.data is a JSON string. Parse it before you use it.

finish_reason "stop" = complete. "length" = cut off: raise max_tokens.

Split a video into timed, tagged segments

Describe the segment types you want and the fields to fill. Pegasus returns each segment's start, end, and your fields.

- Async only: create a task, then poll it
- Set analysis_mode="time_based_metadata"
- No prompt: your descriptions are the instructions
- Up to 20 segment definitions per task
- Tune length with min/max_segment_duration



Playground Segment: each block on the timeline is one segment with its own metadata.

Define segments, get timed JSON back

```
task = client.analyze_async.tasks.create(
  model_name="pegasus1.6",
  video=VideoContext_AssetId(asset_id=asset.id),
  analysis_mode="time_based_metadata",
  response_format=AsyncResponseFormat(
    type="segment_definitions",
    segment_definitions=[{
      "id": "possessions",
      "description": "Each offensive possession",
      "fields": [
        {"name": "result", "type": "string",
         "enum": ["score", "miss", "turnover"],
         "description": "How the possession ended"}]}]))

# poll analyze_async.tasks.retrieve(task.task_id)
```

What you get back

```
{"possessions": [
  {"start_time": 12.0,
   "end_time": 31.5,
   "metadata": {
     "result": "score"}},
  ...]}
```

Field types: string, number, integer, boolean, array, timestamp, time_array. Parse task.result.data when status is "ready".

PART 3 · ASK IT

Jockey

The video agent, knowledge stores, and the Agents API

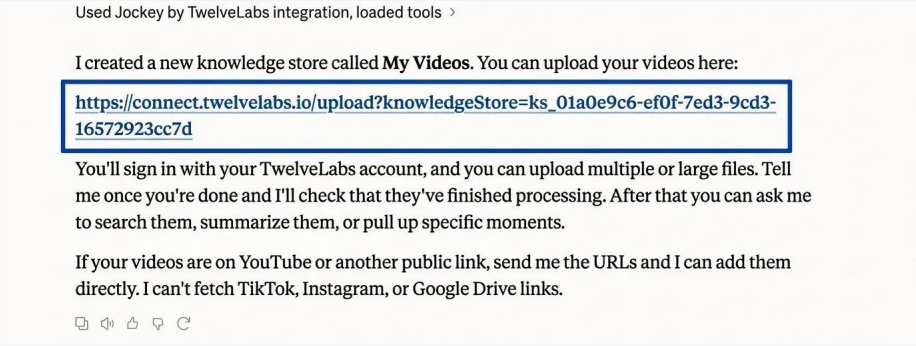
JOCKEY · VIDEO AGENT

Jockey: an agent that reasons over video

Ask in plain language. Jockey plans its own steps with Marengo and Pegasus and answers from every video in a knowledge store. DEMO

- Answers grounded in the footage
- Text or typed JSON (JSON Schema)
- Follow-ups with session_id
- instructions set the lens (scout, editor)
- No code: connect.twelvelabs.io or MCP

Highlight reels · Track a player across games · Tag every play



Jockey inside Claude via MCP: mcp.twelvelabs.io/jockey/mcp

JOCKEY · KNOWLEDGE STORE

A knowledge store is Jockey's memory

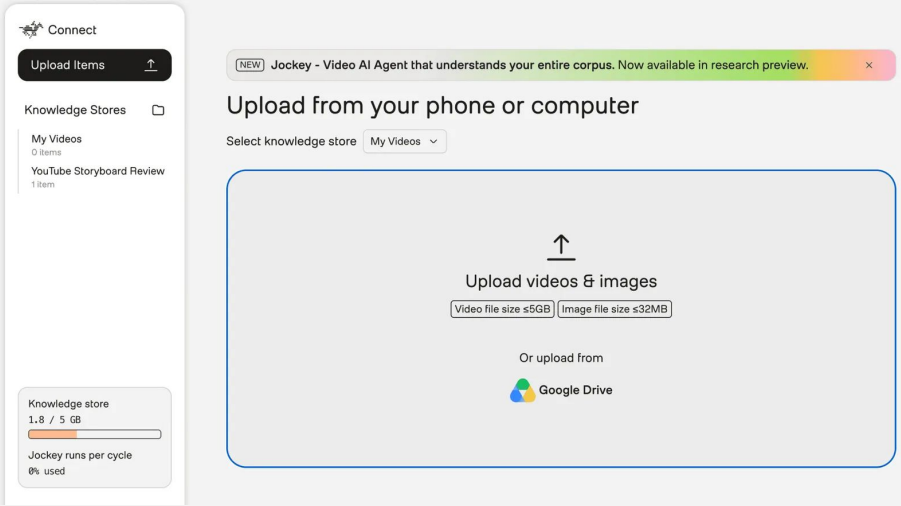
Your media, plus what Jockey learns:

Context: entities, moments, summaries

Ontology: a typed schema linking them

Embeddings: vectors for search

Items: queued → processing → ready. A query hits one store.



Upload and manage stores in Jockey Connect: connect.twelvelabs.io

JOCKEY · AGENTS API

Fill a store, then ask it or search it

Setup: knowledge_stores.create(name) → knowledge_store_items.create(store.id, asset_id) → poll until ready

Ask: a grounded answer (text or JSON)

```
resp = client.responses.create(
  knowledge_store_id=store.id,
  input=[{"type": "message",
          "role": "user",
          "content": "List every 3-pointer"}])

for o in resp.output:
  if o.type == "message":
    print(o.content[0].text)
```

Search: ranked clips with start and end

```
r = client.knowledge_stores.search(
  knowledge_store_id=store.id,
  query={"text": "crowd celebrates"},
  search_options={"video": {
    "modalities": ["visual", "audio"]}})

for hit in r.data:
  m = hit.matches[0]
  print(hit.rank, m.start_sec, m.end_sec)
```

CHEAT SHEET · MODELS API

The Models API on one page

Python calls on client = TwelveLabs(api_key=...) · REST base: https://api.twelvelabs.io/v1.3

Upload media · POST /assets

```
assets.create(method="url", url=...)
```

Create an index · POST /indexes

```
indexes.create(index_name, models=[...])
```

Index an asset · POST /indexes/{id}/indexed-assets

```
indexes.indexed_assets.create(index_id, asset_id)
```

Search moments · POST /search

```
search.query(index_id, query_text, search_options)
```

Create an entity · POST /entity-collections

```
entity_collections.entities.create(id, name,
asset_ids)
```

Embeddings (Marengo 3.5) · POST /embed-v2

```
embed.v_2.create(model_name="marengo3.5", ...)
```

Text or JSON (Pegasus) · POST /analyze

```
analyze(model_name="pegasus1.6", video, prompt)
```

Timed segments · POST /analyze/tasks

```
analyze_async.tasks.create(analysis_mode=...)
```

CHEAT SHEET · AGENTS API (JOCKEY)

The Agents API on one page

Same client, same assets. Upload once, then add the asset_id to a store.

Upload media · POST /assets

```
assets.create(method="url", url=...)
```

Create a store · POST /knowledge-stores

```
knowledge_stores.create(name=...)
```

Add an item · POST /knowledge-stores/{id}/items

```
knowledge_store_items.create(..., asset_id=...)
```

Poll until ready · GET .../items/{item_id}

```
knowledge_store_items.retrieve(...).status
```

Ask a question · POST /responses

```
responses.create(knowledge_store_id, input=[...])
```

Search clips · POST /knowledge-stores/{id}/search

```
knowledge_stores.search(..., query={"text": ...})
```

Extras on responses.create: session_id for follow-ups, instructions for a lens, text=TextParam(...) for JSON.

BEFORE YOU BUILD

Six things that save you an hour

Direct media links only

Use an .mp4 URL or a local file. No YouTube or Drive.

Everything is async

Poll assets, indexing, and store items until ready.

Use a fresh SDK

pip install --upgrade twelvelabs before you start.

Pick the right Marengo

Index search uses 3.0. Use 3.5 for embeddings.

Use Pegasus 1.6

Pass model_name="pegasus1.6" on every analyze call.

Retry the right errors

Retry 429 and 5xx with backoff. Fix other 4xx errors.

RESOURCES

Go build something great

Try it

playground.twelvelabs.io
connect.twelvelabs.io
Jockey MCP: mcp.twelvelabs.io/jockey/mcp

Learn it

docs.twelvelabs.io
AI-ready docs: docs.twelvelabs.io/llms.txt
Docs MCP: docs.twelvelabs.io/_mcp/server

SPURS² HACKATHON · OCT 22–23

Questions?

Assets, Marengo, Pegasus 1.6, Jockey, or the API.

playground.twelvelabs.io · docs.twelvelabs.io